

Minikube Kubernetes Demo Guide

Distributed Document Processing Pipeline — IIT Roorkee CSL-520 Option A · 8 GB RAM · Mock LLM · Ubuntu / Linux

Quick Reference URLs

Service	URL	Notes
Ingestion API Swagger	http://localhost:8000/docs	Upload documents via browser
Query API Swagger	http://localhost:8001/docs	Search & stats via browser
MinIO Console	http://localhost:9001	Login: minioadmin / minioadmin

Step 0 — Prerequisites Check

Run these once to confirm all tools are installed. Skip installs you already have.

```
docker --version          # need 24.x or higher
kubect1 version --client
minikube version
python3 --version         # need 3.8+
```

Install if missing (Ubuntu)

```
# Docker

sudo apt-get update && sudo apt-get install -y docker.io

sudo usermod -aG docker $USER

# Log out and back in after this


# kubectl

sudo snap install kubectl --classic


# minikube

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube


# Python dependencies for the stress test

pip3 install httpx requests
```

Step 1 — Full Cleanup

Run this every time you start the demo from scratch. It removes all stopped containers and unused Docker images — this is intentional.

```
# Stop and delete any existing minikube cluster

minikube stop 2>/dev/null || true
minikube delete 2>/dev/null || true


# Stop docker-compose stack if it was running

cd ~/Desktop/cc-project-main
docker compose --profile llm down -v 2>/dev/null || true
docker compose down -v 2>/dev/null || true


# Free disk space — removes all stopped containers, unused images, cache

docker system prune -af


# Confirm minikube is gone

minikube status

# Expected: 'minikube' profile does not exist
```

Step 2 — Start Minikube

```
minikube start \  
  --memory=5120 \  
  --cpus=4 \  
  --driver=docker \  
  --disk-size=20g \  
  --rootless=false
```

`--rootless=false` **is required on Ubuntu.** Without it minikube throws a rootless driver error.

```
# Verify the cluster is up  
minikube status  
kubectl get nodes  
# Expected: minikube    Ready    control-plane    Xm    v1.xx.x
```

Step 3 — Build Docker Images Inside Minikube

CRITICAL: Run `eval $(minikube docker-env)` in **every new terminal** you open. Without it, images are built in your host Docker and Kubernetes cannot find them.

```
# Point shell at minikube's internal Docker daemon
eval $(minikube docker-env)

# Confirm you are inside minikube's context
docker info | grep 'Name:'
# Should show: Name: minikube

# Go to the project directory
cd ~/Desktop/cc-project-main

# Build all three service images
docker build -t dpp/ingestion-api:1.0.0 ./services/ingestion-api
docker build -t dpp/worker:1.0.0          ./services/worker
docker build -t dpp/query-api:1.0.0       ./services/query-api

# Confirm all three images exist inside minikube
docker images | grep dpp
# Expected:
#   dpp/query-api          1.0.0    ...
#   dpp/worker             1.0.0    ...
#   dpp/ingestion-api      1.0.0    ...
```

Step 4 — Install KEDA

KEDA must be installed before applying your manifests.

```
kubect1 apply --server-side \  
  -f https://github.com/kedacore/keda/releases/download/v2.14.0/keda-2.14.0.yaml  
  
# Wait for KEDA operator to be ready (~60 seconds)  
kubect1 wait --for=condition=ready pod \  
  -l app=keda-operator \  
  -n keda \  
  --timeout=120s  
  
# Verify all three KEDA pods are Running  
kubect1 get pods -n keda  
# Expected:  
#   keda-operator-xxx           1/1   Running  
#   keda-metrics-apiserver-xxx  1/1   Running  
#   keda-admission-webhooks-xxx 1/1   Running
```

If keda-metrics-apiserver shows ErrImagePull

```
eval $(minikube docker-env)  
docker pull ghcr.io/kedacore/keda-metrics-apiserver:2.14.0  
kubect1 delete pod -n keda -l app=keda-metrics-apiserver  
  
# Watch it come back  
kubect1 get pods -n keda --watch
```

Step 5 — Deploy the Pipeline

```
cd ~/Desktop/cc-project-main

# Apply manifests in dependency order
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/kafka.yaml
kubectl apply -f k8s/minio.yaml
kubectl apply -f k8s/elasticsearch.yaml
kubectl apply -f k8s/ingestion-api.yaml
kubectl apply -f k8s/query-api.yaml
kubectl apply -f k8s/worker.yaml
kubectl apply -f k8s/keda-scaledobject.yaml

# Confirm all resources were created
kubectl get all -n dpp
```

Step 6 — Wait for All Pods to be Ready

*Elasticsearch is the slowest — takes 2–3 minutes. **CrashLoopBackOff on worker and ingestion-api during startup is normal.** They restart until Kafka and Elasticsearch are ready. Kubernetes heals them automatically — just wait.*

```
# Watch pods come up in real time
kubectl get pods -n dpp --watch

# Press Ctrl+C once all show Running

# OR block until all are ready (up to 5 minutes)
kubectl wait --for=condition=ready pod \
  --all -n dpp --timeout=300s

# Final status check
kubectl get pods -n dpp
```

Expected final state:

NAME	READY	STATUS	RESTARTS	AGE
document-worker-xxx	1/1	Running	2	3m
elasticsearch-xxx	1/1	Running	0	3m
ingestion-api-xxx	1/1	Running	1	3m
kafka-xxx	1/1	Running	0	3m
minio-xxx	1/1	Running	0	3m
query-api-xxx	1/1	Running	1	3m
zookeeper-xxx	1/1	Running	0	3m

Step 7 — Verify KEDA ScaledObject is READY

```
kubectl get scaledobject -n dpp
# Expected: READY = True    ACTIVE = False
#
# NAME                      ...    READY    ACTIVE
# document-worker-scaler    ...    True     False
```

If **READY** shows **False**, run `kubectl describe scaledobject document-worker-scaler -n dpp | tail -20` to see the error. With the fixed `kafka.yaml` and `keda-scaledobject.yaml` from the previous step, it should be **True** on first boot.

Step 8 — Port Forward Services

Open **3 separate terminals** and keep them running throughout the demo.

Terminal 1 — Ingestion API

```
eval $(minikube docker-env)
kubectl port-forward svc/ingestion-api 8000:8000 -n dpp
```

Terminal 2 — Query API

```
eval $(minikube docker-env)
kubectl port-forward svc/query-api 8001:8001 -n dpp
```

Terminal 3 — MinIO Console (optional)

```
eval $(minikube docker-env)
kubect1 port-forward svc/minio 9001:9001 -n dpp
```

Terminal 4 — Demo terminal (run all commands here)

```
# Verify both APIs are up
curl http://localhost:8000/health
curl http://localhost:8001/health
# Both should return: {"status": "healthy"}
```

Part B — Manual Browser Demo

Step 9 — Create Sample HTML Files

```
cd ~/Desktop/cc-project-main
```

```
echo '<html><body><h1>Cloud Computing</h1>
Cloud computing delivers servers, storage, and software over the internet.
Major providers include Amazon Web Services, Microsoft Azure, and Google Cloud.
The key models are IaaS, PaaS, and SaaS.

' > sample_cloud.html
```

```
echo '
```

Kubernetes Orchestration

```
Kubernetes is an open-source container orchestration platform by Google.
It automates deployment, scaling, and management of containerized applications.
Key concepts include pods, deployments, services, and namespaces.
Brendan Burns and Joe Beda created it in 2014.
```

```
' > sample_k8s.html
```

```
echo '
```

Apache Kafka Streaming

```
<p>Kafka is a distributed event streaming platform built for high throughput.
It is used for building real-time data pipelines and streaming applications.
LinkedIn and Uber use Kafka for their event-driven architectures.</p></body></html>'
> sample_kafka.html
```

Step 10 — Upload Documents

Option A — Via Browser (Swagger UI)

1. Open **http://localhost:8000/docs**
2. Click **POST /upload** → **Try it out**
3. Click **Choose File**, select `sample_cloud.html`
4. Click **Execute** — note the `doc_id` in the response
5. Repeat for `sample_k8s.html` and `sample_kafka.html`

Option B — Via curl

```
# Upload all three documents
curl -X POST http://localhost:8000/upload \
  -F 'file=@sample_cloud.html;type=text/html'

curl -X POST http://localhost:8000/upload \
  -F 'file=@sample_k8s.html;type=text/html'

curl -X POST http://localhost:8000/upload \
  -F 'file=@sample_kafka.html;type=text/html'

# Demo moment — show rejection of unsupported file type (returns 400)
echo "this is a text file" > notes.txt
curl -X POST http://localhost:8000/upload \
  -F 'file=@notes.txt;type=text/plain'

# List all uploaded documents stored in MinIO
curl http://localhost:8000/documents
```

Step 11 — Check Worker Processing

Wait ~30 seconds after uploading, then run:

```
# Show worker logs – you should see documents being processed
kubectl logs -n dpp deployment/document-worker --tail=25

# Show Kafka consumer group lag (LAG column should be 0 = all processed)
kubectl exec -n dpp deployment/kafka -- \
  kafka-consumer-groups \
  --bootstrap-server localhost:29092 \
  --group document-workers \
  --describe

# List Kafka topics
kubectl exec -n dpp deployment/kafka -- \
  kafka-topics \
  --bootstrap-server localhost:29092 \
  --list

# Check document count in Elasticsearch
kubectl exec -n dpp deployment/elasticsearch -- \
  curl -s http://localhost:9200/documents/_count
```

Step 12 — Query API — Search and Stats

Open **<http://localhost:8001/docs>** in your browser to use the Swagger UI, or use curl:

```
# Search by keyword
curl 'http://localhost:8001/search?q=cloud+computing'
curl 'http://localhost:8001/search?q=kubernetes'
curl 'http://localhost:8001/search?q=kafka'

# Filter by topic
curl 'http://localhost:8001/search?topic=kubernetes'

# Pipeline statistics
curl http://localhost:8001/stats

# Top topics, names, and organizations across all documents
curl http://localhost:8001/facets

# Get a specific document by ID (copy a doc_id from search results above)
curl http://localhost:8001/documents/<paste-doc-id-here>

# Paginated search
curl 'http://localhost:8001/search?q=cloud&page=1&size=5'
```

Part C — KEDA Auto-Scaling Demo

KEDA watches the Kafka consumer group lag and scales worker pods when lag exceeds 5 messages per partition. `MOCK_LLM_DELAY=1` is set in `worker.yaml` so each document takes ~1 second to process — this makes the lag build up clearly before the worker can drain it.

Step 13 — Manual Scaling Demo

Run these in **two side-by-side terminals** so the scaling is visible.

Watch Terminal — keep this visible

```
kubectl get pods -n dpp -l app=document-worker --watch
```

Upload Terminal — flood Kafka with parallel uploads

```
# 50 parallel background uploads — all hit Kafka simultaneously
for i in {1..50}; do
    curl -s -X POST http://localhost:8000/upload \
        -F 'file=@sample_cloud.html;type=text/html' > /dev/null &
done
wait
echo "All uploaded"
```

Optional — live monitoring terminal

```
watch -n 2 "kubectl get deployment document-worker -n dpp && echo '---' && \
    kubectl exec -n dpp deployment/kafka -- \
        kafka-consumer-groups --bootstrap-server localhost:29092 \
        --group document-workers --describe"
```

Workers scale from **1** → **up to 3** (capped by 3 Kafka partitions). After processing finishes, wait ~30 seconds — KEDA scales back down to 1 automatically.

Step 14 — Python Stress Test

The stress test fires 20 concurrent uploads using threads, which is more reliable than the shell loop for triggering KEDA. All 100 messages land in Kafka before the worker can drain them.

Terminal 1 — Watch pods scale

```
kubectl get pods -n dpp -l app=document-worker --watch
```

Terminal 2 — Run the functional test first, then the stress test

```
cd ~/Desktop/cc-project-main

# Run all 20 functional tests first (should all pass)
python3 scripts/test_pipeline.py

# Then run the stress test
python3 scripts/stress_test.py --count 100 --concurrency 20 --workers 1
```

Expected stress test output:

```
=====
STRESS TEST - 100 documents, 20 concurrent uploads
  Ingestion API: http://localhost:8000
  Query API:     http://localhost:8001
  Worker pods:   1
=====

[1/3] Generating 100 test documents...
[2/3] Uploading 100 documents (20 concurrent)...
  Uploaded: 100/100
  [OK] Upload complete: 100 ok, 0 errors in 5.2s
  Upload throughput: ~1100 docs/min
[3/3] Waiting for worker processing...
  Processing: 100/100 done - ~60 docs/min - ~100s elapsed
=====

STRESS TEST RESULTS
  Documents uploaded:    100/100
  Upload errors:         0
  Upload throughput:     ~1100 docs/min
  Processing throughput: ~60 docs/min (because MOCK_LLM_DELAY=1)
  Total time:            ~105s
=====
```

While this runs, the Watch Terminal should show workers scaling from 1 → 2 or 3 pods, then back to 1 once the queue drains.

Step 15 — Show Kubernetes Architecture

```
# All resources in the dpp namespace
kubectl get all -n dpp

# Services and their ports
kubectl get svc -n dpp

# KEDA ScaledObject status
kubectl get scaledobject -n dpp

# Worker deployment managed by KEDA
kubectl describe deployment document-worker -n dpp

# Enable metrics server and show live CPU/RAM usage
minikube addons enable metrics-server
kubectl top pods -n dpp

# Namespace overview
kubectl describe namespace dpp

# Describe any specific pod
kubectl describe pod <any-pod-name> -n dpp
```

Step 16 — Teardown

```
# Stop port-forward terminals with Ctrl+C first, then:

# Delete the entire application namespace
kubectl delete namespace dpp

# Remove KEDA
kubectl delete \
  -f https://github.com/kedacore/keda/releases/download/v2.14.0/keda-2.14.0.yaml

# Stop and delete minikube
minikube stop
minikube delete

# Optional: clean all docker images and cache
docker system prune -af
```

Troubleshooting

minikube start fails with rootless error

```
minikube config set rootless false
minikube start --memory=5120 --cpus=4 --driver=docker --disk-size=20g --rootless=false
```

KEDA metrics-apiserver shows ErrImagePull

```
eval $(minikube docker-env)
docker pull ghcr.io/kedacore/keda-metrics-apiserver:2.14.0
kubectl delete pod -n keda -l app=keda-metrics-apiserver
kubectl get pods -n keda --watch
```


KEDA ScaledObject shows READY: False

```
# Check what error KEDA is reporting
kubectl describe scaledobject document-worker-scaler -n dpp | tail -20
kubectl logs -n keda deployment/keda-operator --tail=20
```

With the fixed `kafka.yaml` (full DNS advertised listener) and `keda-scaledobject.yaml` (full DNS bootstrap servers), this should not happen on a fresh deploy.

Pod stuck in CrashLoopBackOff (after dependencies are Running)

```
kubectl describe pod <pod-name> -n dpp
kubectl logs <pod-name> -n dpp

# Restart the deployment
kubectl rollout restart deployment/document-worker -n dpp
```

OOMKilled — out of memory

```
# Replicas are already 1 in the fixed yamls, but if it still OOMs:
kubectl scale deployment ingestion-api --replicas=1 -n dpp
kubectl scale deployment query-api --replicas=1 -n dpp
```

Nuclear reset — start completely fresh from Step 5

```
kubectl delete namespace dpp
# Then repeat from Step 5 onwards
```

Most Useful Commands at a Glance

<code>kubectl get pods -n dpp</code>	<code># See all pods</code>
<code>kubectl get pods -n dpp --watch</code>	<code># Watch pods change</code>
<code>kubectl logs -n dpp deployment/document-worker</code>	<code># Worker logs</code>
<code>kubectl get scaledobject -n dpp</code>	<code># KEDA status</code>
<code>kubectl top pods -n dpp</code>	<code># CPU/RAM usage</code>
<code>kubectl describe pod <name> -n dpp</code>	<code># Debug a specific pod</code>